EduForge GenUI — Real User Flow

System Overview

EduForge is an AI-powered adaptive learning platform where **teachers upload presentations** and **students learn through BKT-driven, scaffold-aware AI visualizations**. The core innovation is the neuro-symbolic BKT→GenUI fusion: the Bayesian Knowledge Tracing posterior directly constrains which UI components the LLM can generate.

```
Teacher uploads PPT → Async ingestion pipeline → Subtopics + MCQs generated
Student answers MCQ → BKT updates P(mastery) → Scaffold level (0-4) resolves
→ Allowed components[] sent to Gemini → AI generates constrained UI → React renders
```

Architecture

Service	Port	Role
Next.js Frontend	3000	Teacher/Student UI, SSR, API routing
API Gateway (FastAPI)	8000	Auth proxy, lesson CRUD, enrollment, analytics
BKT Service (FastAPI)	8001	Bayesian Knowledge Tracing engine, scaffold resolver
GenUI Service (FastAPI)	8002	Gemini-powered visualization & Socratic tutor streaming
Ingestion Worker	8003	Pub/Sub-triggered PPT→subtopics→MCQ pipeline

Data stores: Firestore (native), GCS buckets, Pub/Sub
Auth: Firebase Authentication (email/password) + JWT tokens
AI: Google Gemini 2.5 Flash for all LLM tasks

Flow 1: Teacher — Create & Publish Lesson

1.1 Registration

1. Teacher navigates to `/register`
2. Fills email, password, name, selects role "**Teacher**"
3. `sessionStore.register()` → Firebase `createUserWithEmailAndPassword()`
4. Firestore document created: `users/{uid}` with `role: "teacher"`
5. Cookies set: `user-role=teacher`, `firebase-token={jwt}`
6. Redirect → `/dashboard`

1.2 Upload Presentation

1. Teacher clicks "**+ New Lesson**" → `/lessons/new`
2. Fills lesson title, subject, selects `.pptx` file
3. Frontend calls `POST /lessons/upload-url` with `{ filename, contentType, lessonTitle, subject }`
4. API creates lesson doc in Firestore (`status: "draft"`) and returns a signed GCS upload URL
5. Frontend uploads the PPTX directly to GCS via the signed URL (client-side)
6. Frontend calls `POST /lessons/start-ingestion` with `{ lessonId, gcsPath }`
7. API publishes message to Pub/Sub topic `lesson-ingestion-requests`
8. Redirect → `/lessons/{id}` (detail page)

1.3 Ingestion Pipeline (Async)

The ingestion worker processes the PPTX through 7 steps, updating Firestore at each step. The teacher sees real-time progress via Firestore `onSnapshot()` listener.

Step	Action	Firestore Update
1	**Extract** slides from PPTX using `python-pptx`	`progress: 10%`
2	**Layout Parse** via Cloud Document AI	`progress: 25%`
3	**Topic Generation** — Gemini clusters slides into subtopics	`progress: 45%`
4	**MCQ Generation** — Gemini creates 15 MCQs per subtopic at 3 tiers	`progress: 65%`
5	**Prerequisite Graph** — Gemini builds concept dependency graph	`progress: 80%`
6	**BKT Seed** — Generate initial BKT parameters per concept	`progress: 90%`
7	**Complete** — All data written to Firestore subcollections	`progress: 100%, step: "complete"`

1.4 Publish Lesson

1. Once ingestion is complete, teacher sees subtopic count and MCQ count
2. Teacher clicks "**Publish**" button
3. Frontend calls `PATCH /lessons/{id}/publish`
4. API verifies ownership + ingestion complete, updates `status: "published"`
5. A "**Share with students**" box appears with the lesson ID and a "**Copy ID**" button
6. Teacher shares the lesson ID with students (out-of-band: email, LMS, etc.)

1.5 Monitor Students (Analytics)

1. Teacher navigates to lesson detail → analytics tab
2. Frontend calls `GET /analytics/class/{lessonId}`
3. API returns aggregated BKT states for all enrolled students
4. **ClassHeatmap** renders students × concepts color-coded by `P(mastery)`

Flow 2: Student — Enroll & Learn

2.1 Registration

1. Student navigates to `/register`
2. Fills email, password, name, selects role "**Student**"
3. Same Firebase auth flow as teacher, but `role: "student"`
4. Redirect → `/learn`

2.2 Enroll in Lesson

1. Student enters the lesson ID shared by teacher into the enrollment input
2. Frontend calls `POST /students/enroll` with `{ lessonId }`
3. API verifies lesson exists and is `published`
4. Creates enrollment doc: `enrollments/{studentId}_{lessonId}`
5. Lesson appears in student's lesson list
6. If enrollment fails (invalid ID, unpublished), error message shown inline

2.3 Browse Lesson Overview

1. Student clicks a lesson → `/learn/{lessonId}`
2. Frontend loads subtopics via `GET /lessons/{lessonId}/subtopics`
3. Each subtopic card shows:
   - Title, description, difficulty badge (foundational/intermediate/advanced)
   - Key concepts as tags
   - **Mastery progress bar** — computed from BKT states for that subtopic's concepts
4. Student clicks a subtopic → `/learn/{lessonId}/{subtopicId}`

2.4 Core Learning Loop (MCQ + BKT + GenUI)

This is the heart of EduForge — the adaptive learning experience:

```
┌──────────────────────────────────────────────────────────┐
│                      LEARNING PAGE                       │
│                                                          │
│  ┌──────────────────────┐  ┌──────────────────────────┐  │
│  │   AI VISUALIZATION   │  │      MASTERY HUD         │  │
│  │                      │  │                          │  │
│  │  [StepByStep]        │  │  concept_a  ▓▓▓▓ 60%     │  │
│  │  [HintCard]          │  │  concept_b  ▓▓▒▒ 35%     │  │
│  │  [FormulaCard]       │  │  concept_c  ▓▒▒▒ 20%     │  │
│  │                      │  │                          │  │
│  └──────────────────────┘  │  Scaffold: Level 1       │  │
│                            │  (Developing)            │  │
│  ┌──────────────────────┐  └──────────────────────────┘  │
│  │   MCQ QUESTION       │                                │
│  │   Q2 of 15           │                                │
│  │                      │                                │
│  │   [A] option_a       │                                │
│  │   [B] option_b  ←    │                                │
│  │   [C] option_c       │                                │
│  │   [D] option_d       │                                │
│  │                      │                                │
│  │   [Next Question →]  │                                │
│  └──────────────────────┘                                │
└──────────────────────────────────────────────────────────┘
```

Step-by-step:

1. **Page loads**: MCQs fetched via `GET /lessons/{lessonId}/subtopics/{subtopicId}/mcqs`
2. **Initial GenUI generation**: `useGenUI` hook calls `POST /api/genui` with the first concept
3. **GenUI server route** orchestrates:
   - Fetches BKT state → `GET /bkt/state?studentId=X&conceptId=Y`
   - Resolves scaffold level → `GET /bkt/scaffold?p_mastery=0.2`
   - Streams visualization → `POST /genui-service/stream/visualize`
4. **Gemini generates** a constrained JSON array of UI components based on scaffold level
5. **Frontend validates** components against `ALLOWED_MAP[scaffoldLevel]` and renders them
6. **Student reads visualization**, then answers the MCQ

On MCQ answer:

7. Frontend calls `POST /bkt/update` with:

```
{
  "student_id": "...",
  "concept_id": "...",
  "is_correct": true/false,
  "time_taken_seconds": 12
}
```

8. **BKT Engine** runs Bayesian update:
   - If correct: `P(mastery) increases` based on `P(T)` and `1 - P(S)`
   - If wrong: `P(mastery) may decrease` based on `P(G)`
   - Tracks `consecutiveCorrect`, `consecutiveWrong`, `totalAttempts`
   - If 3+ consecutive wrong → **misconception detected**
9. **Scaffold Resolver** maps new `P(mastery)` to level 0-4:
   - Level 0 (Novice): `P < 0.2` — StepByStep, HintCard, FormulaCard, AnalogyCard
   - Level 1 (Developing): `0.2 ≤ P < 0.4` — StepByStep, HintCard, FormulaCard, ConceptDiagram
   - Level 2 (Approaching): `0.4 ≤ P < 0.6` — ConceptDiagram, FormulaCard, HintCard, PracticeExercise
   - Level 3 (Proficient): `0.6 ≤ P < 0.8` — ConceptDiagram, PracticeExercise, ProofWalkthrough
   - Level 4 (Mastered): `P ≥ 0.8` — ConceptDiagram, ExpertSummary, ProofWalkthrough, PracticeExercise
10. If scaffold level changed → **GenUI regenerates** with new component constraints
11. **MasteryHUD updates** in real-time showing per-concept progress bars
12. Student proceeds to next MCQ → repeat from step 6
13. After all MCQs → "**Back to Lesson**" link to return and pick next subtopic

2.5 Mastery Threshold

- When `P(mastery) ≥ 0.95` for a concept → marked as **mastered**
- When all concepts in a subtopic reach mastery → subtopic complete
- Prerequisite subtopics must reach `P(mastery) > 0.6` before dependent subtopics unlock

---

The 8 GenUI Components

Component	Purpose	Available at Level
StepByStep	Numbered step breakdown with examples	0, 1
HintCard	Gentle/moderate/direct hints	0, 1, 2
FormulaCard	Mathematical formulas with variables explained	0, 1, 2
AnalogyCard	Real-world analogies to explain concepts	0
ConceptDiagram	Hierarchy/flow/comparison diagrams	1, 2, 3, 4
PracticeExercise	Interactive practice problems	2, 3, 4
ProofWalkthrough	Formal proof steps with justifications	3, 4
ExpertSummary	Dense expert-level overviews	4

Key constraint: The BKT posterior is a *hard constraint* on LLM generation. A Level 0 student will never see ProofWalkthrough or ExpertSummary, regardless of what the LLM would prefer to generate. This is enforced at the prompt level (system prompt includes allowed/forbidden components) and at the frontend level (validation filter).

Data Flow Diagram

```
TEACHER                              STUDENT
  │                                    │
  ├── Register ──→ Firebase Auth       ├── Register ──→ Firebase Auth
  │                                    │
  ├── Upload PPT ──→ GCS               ├── Enroll ──→ API → Firestore
  │       │                            │
  │       ├── Start Ingestion          ├── View Subtopics
  │       │       │                    │       │
  │       │     Pub/Sub                │       │
  │       │       │                    │       ▼
  │       │    Ingestion Worker        ├── Open Subtopic
  │       │       │                    │       │
  │       │       ├── Extract          │       ├── Load MCQs
  │       │       ├── Parse            │       ├── Generate GenUI
  │       │       ├── Topics           │       │       │
  │       │       ├── MCQs             │       │    ┌──┴──┐
  │       │       ├── Graph            │       │    BKT  GenUI
  │       │       ├── BKT Seed         │       │    Svc  Svc
  │       │       └── Complete         │       │    └──┬──┘
  │       │                            │       │       │
  │    Firestore ←──────────────────────────────────────┘
  │       │                            │       │
  ├── Publish ──→ API                  │       ├── Answer MCQ
  │                                    │       │       │
  ├── Share ID ──→ (out-of-band)       │       │    BKT Update
  │                                    │       │       │
  └── View Analytics                   │       │    Scaffold → GenUI Regen
                                       │       │       │
                                       │       └── Next Question
                                       │
                                       └── Complete → Back to Lesson
```

Authentication Flow

1. User enters email/password on `/login` or `/register`
2. Firebase Auth returns JWT token
3. Token stored in Zustand store + cookies (`firebase-token` , `user-role`)
4. Next.js middleware reads `user-role` cookie for route protection:
   - `/dashboard` , `/lessons/*` → requires `teacher`
   - `/learn/*` → requires `student`
5. Layout components (`(teacher)/layout.tsx` , `(student)/layout.tsx`) provide auth guards + navigation
6. `ClientProviders` component auto-refreshes token every 50 minutes
7. All API calls include `Authorization: Bearer {token}` header

Key Technical Decisions

Decision	Rationale
Firebase Auth (not custom JWT)	Fastest path to production auth with email/password
Firestore real-time listeners	Ingestion progress updates without polling
Pub/Sub for ingestion	Decouples upload from heavy processing; enables retry
BKT as separate service	Stateless math engine; scales independently
GenUI as SSE stream	Progressive rendering; user sees components appear
Frontend component validation	Defense-in-depth; LLM can't bypass scaffold rules
Zustand (not Redux)	Minimal boilerplate for 2-store app
Server-side GenUI route	Keeps service URLs private; adds error handling layer